# E-Commerce Order Management Database System

Week 3 – Designing the Database Structure

Relationships · ER Diagrams · Relational Schema

**Course:** Database Management Systems (DBMS) **Name:** _____

**Project:** E-Commerce Order Management Database System **Roll No.:** _____

## 1. Objective

This document designs the database structure of an online shopping platform (similar to Amazon or Flipkart). Starting from the eight entities identified in Week 2, it (a) identifies the **one-to-one, one-to-many and many-to-many relationships** between them, (b) presents the complete **Entity Relationship (ER) diagram** in two notations – **Chen notation** and **Crow's Foot (MySQL Workbench style)** – and (c) converts the ER model into a **relational schema** that will be used to create the database tables in the next stage.

## 2. Entities and Their Attributes

The Week 2 entities are Customer, Product, Category, Supplier, Order, Payment, Shipment and Review. Resolving the two many-to-many relationships (Section 3.4) adds two associative entities, **Order_Item** and **Product_Supplier**. The Order entity is named **Orders** in the schema because **ORDER** is a reserved word in SQL.

| Entity | Purpose | Primary key | Attributes |
|---|---|---|---|
| Customer | A registered buyer | customer_id | first_name, last_name, email, phone, address, city, state, pincode, registered_at |
| Category | Group of similar products (e.g. Electronics) | category_id | category_name, description |
| Supplier | Vendor that supplies products | supplier_id | supplier_name, contact_person, email, phone, city |
| Product | Item that can be sold | product_id | product_name, description, price, stock_quantity |
| Orders | A purchase placed by a customer | order_id | order_date, order_status, total_amount |
| Payment | Payment made for an order | payment_id | payment_date, payment_method, payment_status, amount |
| Shipment | Delivery of an order | shipment_id | carrier, tracking_number, shipped_date, delivery_date, shipment_status, shipping_address |
| Review | Rating and comment on a product | review_id | rating, comment, review_date |
| Order_Item *(associative)* | Products inside an order | (order_id, product_id) | quantity, unit_price |
| Product_Supplier *(associative)* | Which supplier supplies which product | (product_id, supplier_id) | supply_price |

# 3. Relationships Between Entities

Each relationship below is read in both directions (e.g. *one customer places many orders; each order is placed by exactly one customer*). Participation tells whether every instance must take part: **total** = mandatory, **partial** = optional.

## 3.1 Relationship summary

| # | Name | Entities | Type | Participation | Business rule |
|---|---|---|---|---|---|
| 1 | Places | Customer – Orders | 1 : N | Orders: total<br>Customer: partial | A customer can place many orders; every order belongs to exactly one customer. |
| 2 | Belongs_To | Category – Product | 1 : N | Product: total<br>Category: partial | A category groups many products; every product is in exactly one category. |
| 3 | Writes | Customer – Review | 1 : N | Review: total<br>Customer: partial | A customer can write many reviews; each review is written by one customer. |
| 4 | Has | Product – Review | 1 : N | Review: total<br>Product: partial | A product can receive many reviews; each review is about one product. |
| 5 | Paid_By | Orders – Payment | 1 : 1 | Payment: total<br>Orders: partial | One order is settled by one payment record; each payment belongs to one order. |
| 6 | Shipped_Via | Orders – Shipment | 1 : 1 | Shipment: total<br>Orders: partial | One order has one shipment record; each shipment carries one order. |
| 7 | Contains | Orders – Product | M : N | Both partial | An order contains many products; a product appears in many orders. Resolved by **Order_Item**. |
| 8 | Supplies | Supplier – Product | M : N | Both partial | A supplier supplies many products; a product can come from many suppliers. Resolved by **Product_Supplier**. |

## 3.2 One-to-one (1 : 1) relationships

In a 1 : 1 relationship one instance of A is linked to at most one instance of B and vice-versa.

• **Orders – Payment:** Order #101 has exactly one Payment record (e.g. UPI, ₹1,499, status ‘Success’), and that payment belongs only to Order #101.
• **Orders – Shipment:** Order #101 is delivered in one Shipment (tracking no. TRK98231, carrier ‘BlueDart’), and that shipment carries only Order #101.

## 3.3 One-to-many (1 : N) relationships

In a 1 : N relationship one instance of A is linked to many instances of B, but each B instance belongs to only one A.

• **Customer – Orders:** Customer #5 (Riya) has placed Orders #101, #108 and #120.
• **Category – Product:** Category ‘Mobiles’ contains Products ‘Redmi 13’, ‘Galaxy M35’ and ‘iPhone 15’.
• **Customer – Review:** Customer #5 has written Reviews #1, #2 and #3 for different products.
• **Product – Review:** Product ‘Redmi 13’ has received Reviews #1, #7 and #12 from different customers.

## 3.4 Many-to-many (M : N) relationships

An M : N relationship cannot be stored directly in a relational table. It is resolved with an **associative (junction) table** that holds the primary keys of both entities as foreign keys, turning one M : N into two 1 : N relationships.

**Orders – Product → Order_Item.** Order #101 contains Products 5 and 7; Product 5 also appears in Order #102.

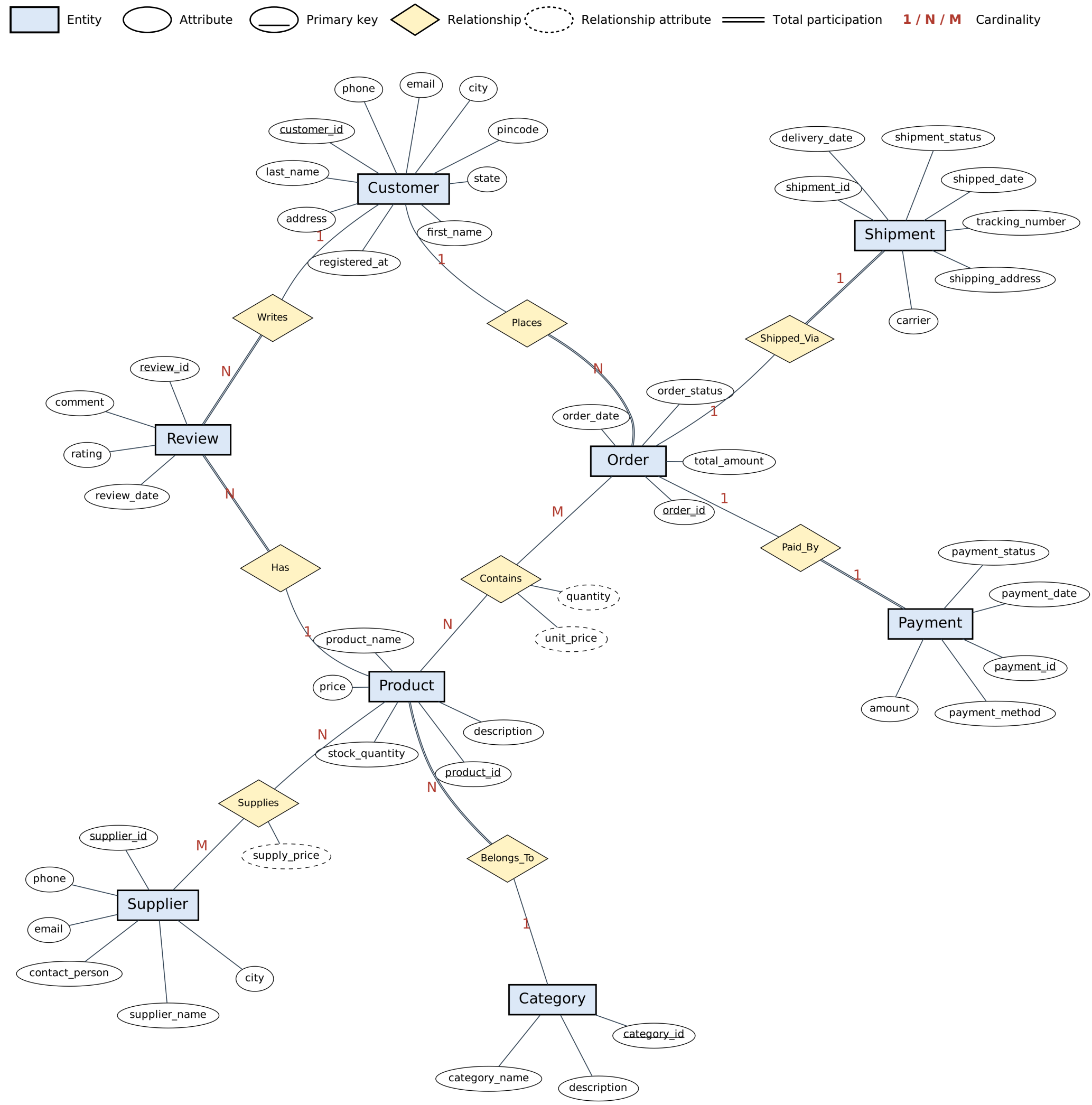
| order_id (FK) | product_id (FK) | quantity | unit_price |
|---|---|---|---|
| 101 | 5 | 2 | 499.00 |
| 101 | 7 | 1 | 501.00 |
| 102 | 5 | 1 | 499.00 |

**Product – Supplier → Product_Supplier.** Product 5 is supplied by Suppliers 2 and 3; Supplier 2 also supplies Product 7.

| product_id (FK) | supplier_id (FK) | supply_price |
|---|---|---|
| 5 | 2 | 410.00 |
| 5 | 3 | 425.00 |
| 7 | 2 | 455.00 |

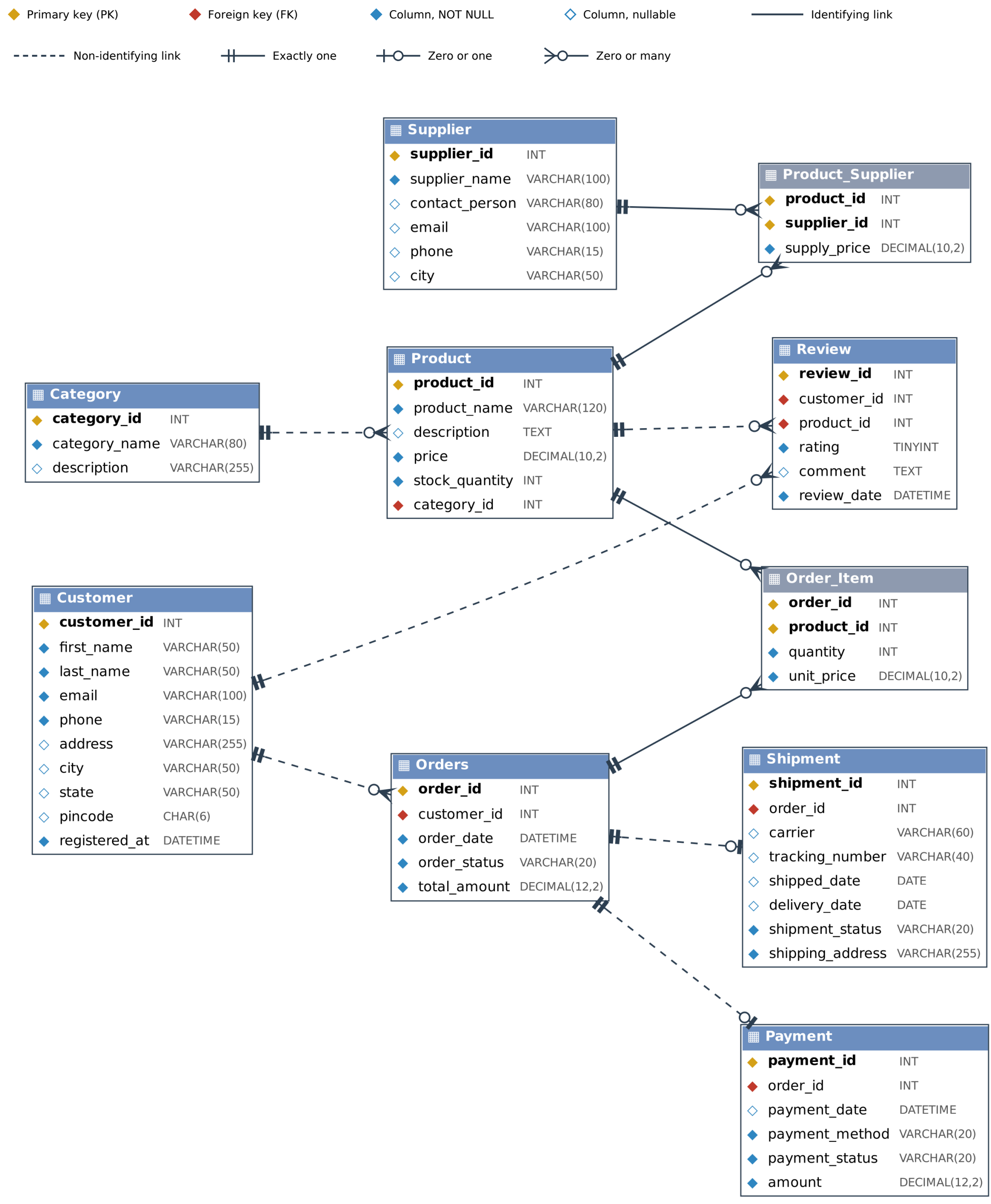
The attributes *quantity*, *unit_price* and *supply_price* describe the relationship itself, so they are stored in the junction tables.

## 4. ER Diagram 1 – Chen Notation (Conceptual Model)




Foreign-key columns are not drawn as attributes in the Chen diagram; they are represented by the relationship diamonds and appear in the relational schema (Section 7). Cardinality labels: Customer 1 – N Orders, Category 1 – N Product, Customer 1 – N Review, Product 1 – N Review, Orders 1 – 1 Payment, Orders 1 – 1 Shipment, Orders M – N Product, Supplier M – N Product.

## 5. ER Diagram 2 – Crow's Foot Notation (MySQL Workbench Style EER Diagram)



# 6. Converting the ER Model into a Relational Schema

The following standard mapping rules were applied to the ER model:

| ER construct | Rule applied | Result in this project |
|---|---|---|
| Strong entity | Each entity becomes a table; its attributes become columns; the key attribute becomes the primary key. | Customer, Category, Supplier, Product, Orders, Payment, Shipment, Review |
| 1 : N relationship | Add the primary key of the 'one' side as a foreign key in the table on the 'many' side. | Orders.customer_id, Product.category_id, Review.customer_id, Review.product_id |
| 1 : 1 relationship | Place a foreign key in the table with total participation and add a UNIQUE constraint so only one row can match. | Payment.order_id (UNIQUE), Shipment.order_id (UNIQUE) |
| M : N relationship | Create a new junction table. Its primary key is the combination of both foreign keys. | Order_Item (order_id, product_id), Product_Supplier (product_id, supplier_id) |
| Relationship attribute | The attribute moves into the junction table as a normal column. | quantity, unit_price, supply_price |
| Total participation | The foreign key is declared NOT NULL. | Orders.customer_id, Product.category_id, Review.*_id, Payment.order_id, Shipment.order_id |

# 7. Relational Schema

## 7.1 Schema in relational notation

Underlined = primary key, *italic* = foreign key (the referenced table is shown after the arrow).

**Customer** ( customer_id, first_name, last_name, email, phone, address, city, state, pincode, registered_at )

**Category** ( category_id, category_name, description )

**Supplier** ( supplier_id, supplier_name, contact_person, email, phone, city )

**Product** ( product_id, product_name, description, price, stock_quantity, *category_id* )

**Product_Supplier** ( *product_id*, *supplier_id*, supply_price )

**Orders** ( order_id, *customer_id*, order_date, order_status, total_amount )

**Order_Item** ( *order_id*, *product_id*, quantity, unit_price )

**Payment** ( payment_id, *order_id*, payment_date, payment_method, payment_status, amount )

**Shipment** ( shipment_id, *order_id*, carrier, tracking_number, shipped_date, delivery_date, shipment_status, shipping_address )

**Review** ( review_id, *customer_id*, *product_id*, rating, comment, review_date )

**Foreign keys:** Product.category_id → Category | Product_Supplier.product_id → Product | Product_Supplier.supplier_id → Supplier | Orders.customer_id → Customer | Order_Item.order_id → Orders | Order_Item.product_id → Product | Payment.order_id → Orders | Shipment.order_id → Orders | Review.customer_id → Customer | Review.product_id → Product

## 7.2 Table definitions

Data types follow MySQL so the schema can be used directly to create the tables. PK = primary key, FK = foreign key, NN = NOT NULL, UQ = UNIQUE.

### 7.2.1 Customer

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **customer_id** | INT | PK | |
| first_name | VARCHAR(50) | NN | |
| last_name | VARCHAR(50) | NN | |
| email | VARCHAR(100) | NN, UQ | Used as login ID |
| phone | VARCHAR(15) | NN | |
| address | VARCHAR(255) | | |
| city | VARCHAR(50) | | |
| state | VARCHAR(50) | | |
| pincode | CHAR(6) | | |
| registered_at | DATETIME | NN | Default: current time |

### 7.2.2 Category

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **category_id** | INT | PK | |
| category_name | VARCHAR(80) | NN, UQ | |
| description | VARCHAR(255) | | |

### 7.2.3 Supplier

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **supplier_id** | INT | PK | |
| supplier_name | VARCHAR(100) | NN | |
| contact_person | VARCHAR(80) | | |
| email | VARCHAR(100) | UQ | |
| phone | VARCHAR(15) | | |
| city | VARCHAR(50) | | |

### 7.2.4 Product

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **product_id** | INT | PK | |
| product_name | VARCHAR(120) | NN | |
| description | TEXT | | |
| price | DECIMAL(10,2) | NN | CHECK (price > 0) |
| stock_quantity | INT | NN | CHECK (≥ 0), default 0 |
| category_id | INT | FK, NN | → Category |

### 7.2.5 Product_Supplier

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **product_id** | INT | PK, FK | → Product |
| **supplier_id** | INT | PK, FK | → Supplier |
| supply_price | DECIMAL(10,2) | NN | |

### 7.2.6 Orders (the Order entity)

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **order_id** | INT | PK | |
| customer_id | INT | FK, NN | → Customer |
| order_date | DATETIME | NN | Default: current time |
| order_status | VARCHAR(20) | NN | Placed / Confirmed / Shipped / Delivered / Cancelled |
| total_amount | DECIMAL(12,2) | NN | |

### 7.2.7 Order_Item

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **order_id** | INT | PK, FK | → Orders |
| **product_id** | INT | PK, FK | → Product |
| quantity | INT | NN | CHECK (quantity > 0) |
| unit_price | DECIMAL(10,2) | NN | Price at time of order |

### 7.2.8 Payment

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **payment_id** | INT | PK | |
| order_id | INT | FK, NN, UQ | → Orders; 1 : 1 |
| payment_date | DATETIME | | |
| payment_method | VARCHAR(20) | NN | UPI / Card / NetBanking / COD |
| payment_status | VARCHAR(20) | NN | Pending / Success / Failed / Refunded |
| amount | DECIMAL(12,2) | NN | |

### 7.2.9 Shipment

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **shipment_id** | INT | PK | |
| order_id | INT | FK, NN, UQ | → Orders; 1 : 1 |
| carrier | VARCHAR(60) | | |
| tracking_number | VARCHAR(40) | UQ | |
| shipped_date | DATE | | |
| delivery_date | DATE | | |
| shipment_status | VARCHAR(20) | NN | Packed / In Transit / Out for Delivery / Delivered |
| shipping_address | VARCHAR(255) | NN | |

### 7.2.10 Review

| Column | Data type | Key / constraint | Notes |
|---|---|---|---|
| **review_id** | INT | PK | |
| customer_id | INT | FK, NN | → Customer |
| product_id | INT | FK, NN | → Product |
| rating | TINYINT | NN | CHECK (1 to 5) |
| comment | TEXT | | |
| review_date | DATETIME | NN | Default: current time |
| *Table constraint* | | UQ (customer_id, product_id) | One review per customer per product |

## 7.3 Foreign key summary

| Child table.column | Parent table.column | Relationship | ON DELETE | Reason |
|---|---|---|---|---|
| Product.category_id | Category.category_id | 1 : N | RESTRICT | A category in use cannot be removed |
| Orders.customer_id | Customer.customer_id | 1 : N | RESTRICT | Order history must be kept |
| Review.customer_id | Customer.customer_id | 1 : N | CASCADE | Reviews go with the customer account |
| Review.product_id | Product.product_id | 1 : N | CASCADE | Reviews go with the product |
| Payment.order_id | Orders.order_id | 1 : 1 | RESTRICT | Payment records must be kept |
| Shipment.order_id | Orders.order_id | 1 : 1 | CASCADE | Shipment belongs to the order |
| Order_Item.order_id | Orders.order_id | 1 : N (M : N) | CASCADE | Items belong to the order |
| Order_Item.product_id | Product.product_id | 1 : N (M : N) | RESTRICT | Sold products cannot be deleted |
| Product_Supplier.product_id | Product.product_id | 1 : N (M : N) | CASCADE | Link removed with the product |
| Product_Supplier.supplier_id | Supplier.supplier_id | 1 : N (M : N) | CASCADE | Link removed with the supplier |

ON UPDATE CASCADE is applied to all foreign keys. The ON DELETE rules above are design choices that will be written into the CREATE TABLE statements.

## 7.4 Integrity rules

• **Entity integrity:** every table has a primary key (single column, or composite in Order_Item and Product_Supplier) that is never NULL.
• **Referential integrity:** every foreign key value must exist in the parent table, or be rejected.
• **Uniqueness:** Customer.email, Category.category_name, Supplier.email, Shipment.tracking_number, Payment.order_id, Shipment.order_id and (customer_id, product_id) in Review.
• **Domain rules:** price > 0, quantity > 0, stock_quantity ≥ 0, rating between 1 and 5, and status columns restricted to the listed values.

# 8. Summary

The design has **8 entities plus 2 associative tables**, giving **10 relational tables**. It contains **2 one-to-one** (Orders–Payment, Orders–Shipment), **4 one-to-many** (Customer–Orders, Category–Product, Customer–Review, Product–Review) and **2 many-to-many** relationships (Orders–Product, Supplier–Product), the last two implemented through Order_Item and Product_Supplier. The schema is ready to be turned into CREATE TABLE statements in the next stage.